

Lecture 3:

Symmetric Cryptography

Eleftheria Makri | Security

25/02/2026



Universiteit
Leiden

Symmetric Cryptography

- Definition of Cryptology = Cryptography + Cryptanalysis
- Security Goals related to Cryptography
- Classification of Cryptography
- Introduction to Symmetric(-Key) Cryptography
 - Underlying Basic Ideas
 - Stream Ciphers
 - Block Ciphers
- Hash Functions

Cryptography



0101101000011011
10011010100010
1011010101001111
1101010101100010111
1010101010011110101
1101010111101000110
1101010100010011110
1010110101010011011

Cryptanalysis



Cryptology =

Cryptography



+

Cryptanalysis



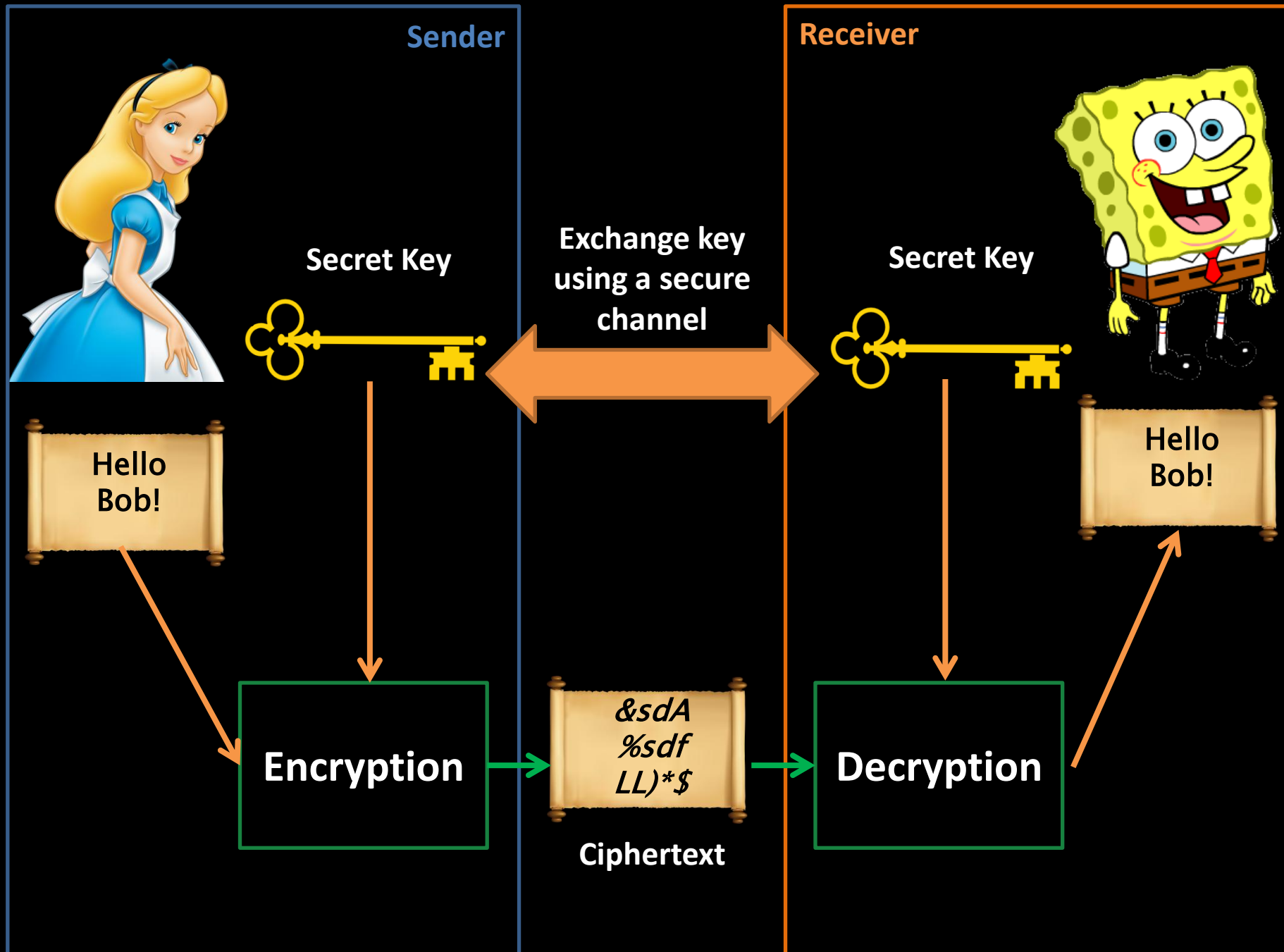
Important goals of cryptography

- Confidentiality = keeping information secret for everyone except for authorized entities
- Authentication
 - of entities = assuring the source of information
 - of data = assuring that the information has not been altered by unauthorized entities
- Non-repudiation = assuring that actions or commitments in the past cannot be denied

Classification of cryptography

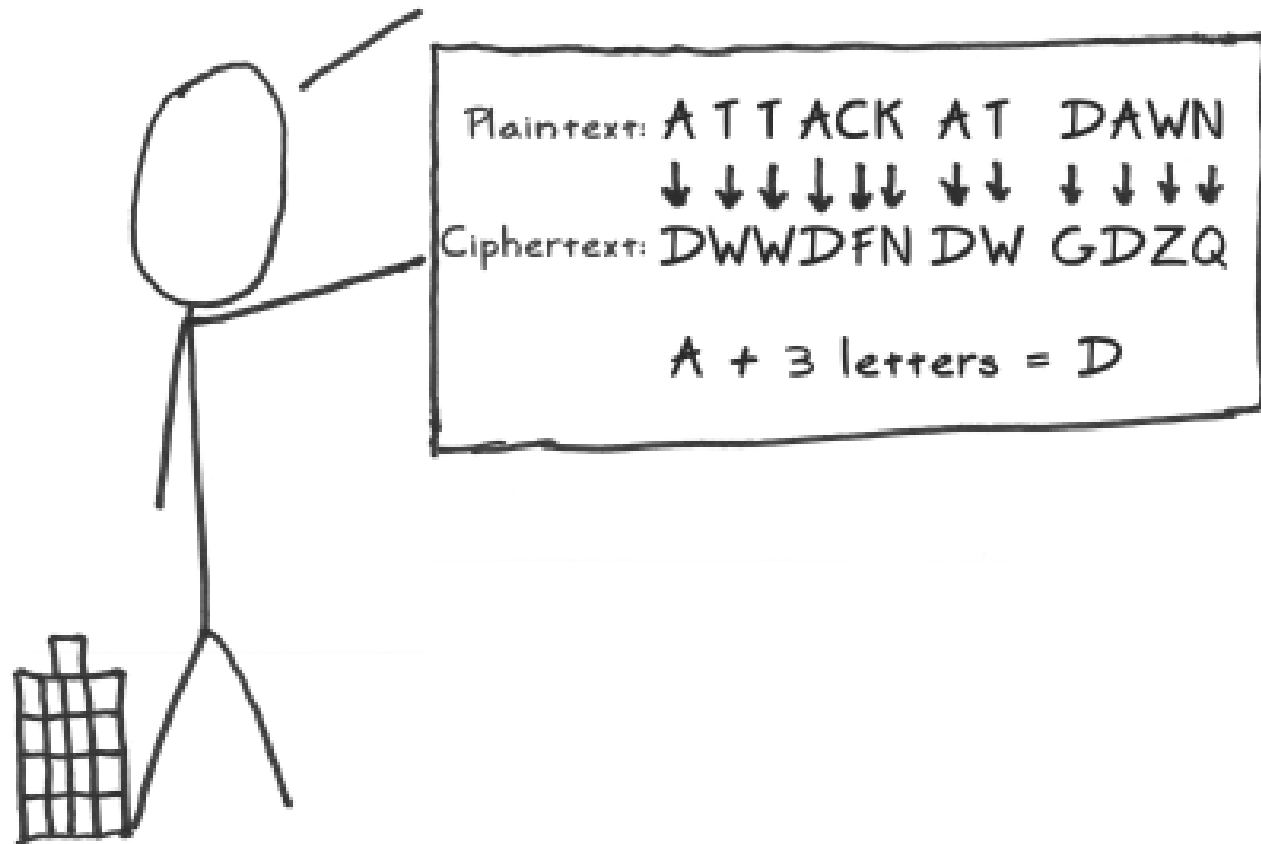
- Symmetric-key cryptography:
 - Mainly for confidentiality
 - Sender and receiver have the same secret key
 - Block ciphers and stream ciphers
- Public-key cryptography:
 - Mainly for authentication (digital signatures) and key agreement
 - Sender and receiver have a different key (= public and private key)
- Hash functions:
 - Mainly for the authentication of data
 - With or without secret key

Symmetric-key cryptography

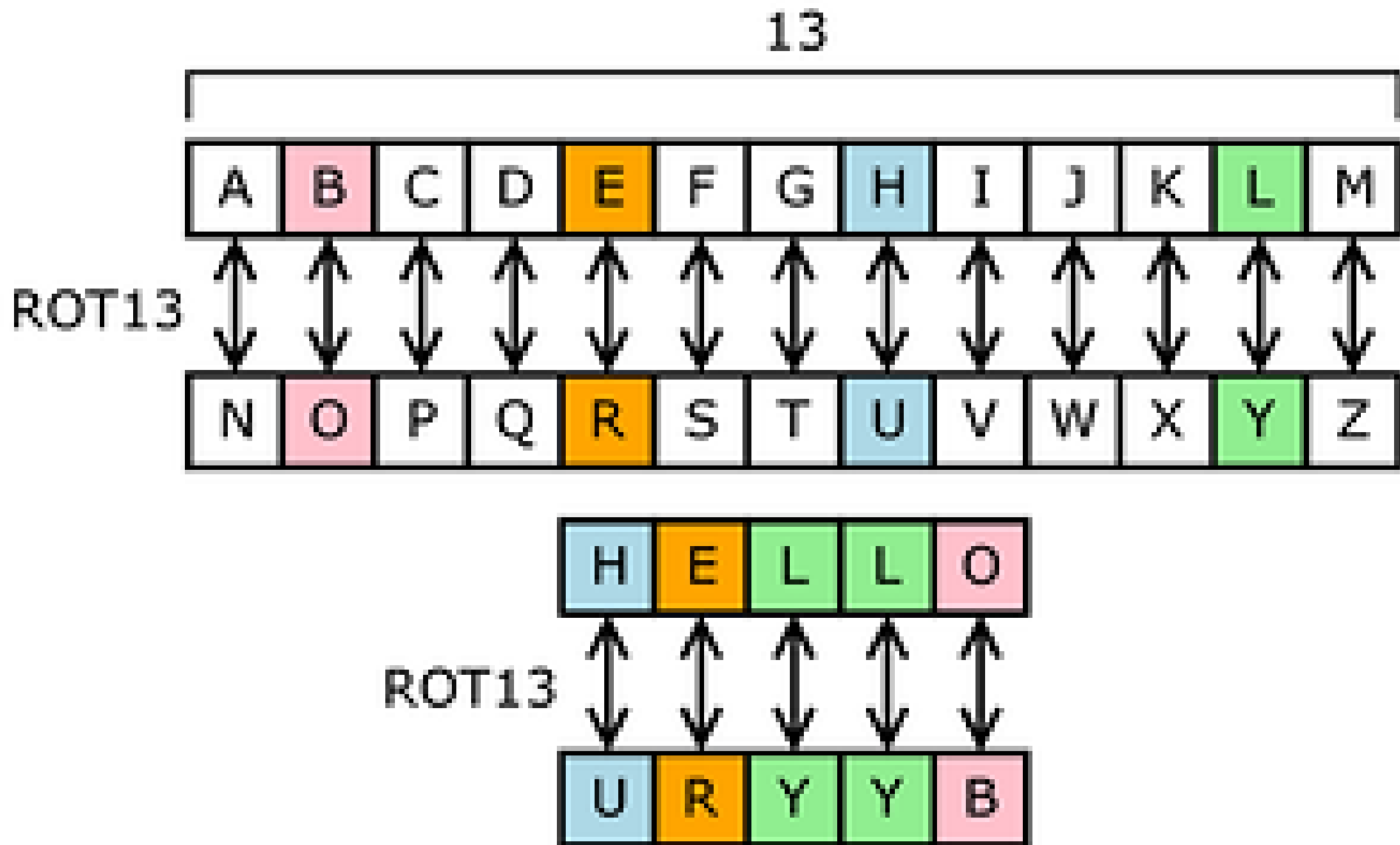


Big Idea #1: Confusion

It's a good idea to obscure the relationship between your real message and your 'encrypted' message. An example of this 'confusion' is the trusty ol' Caesar Cipher:



Substitution

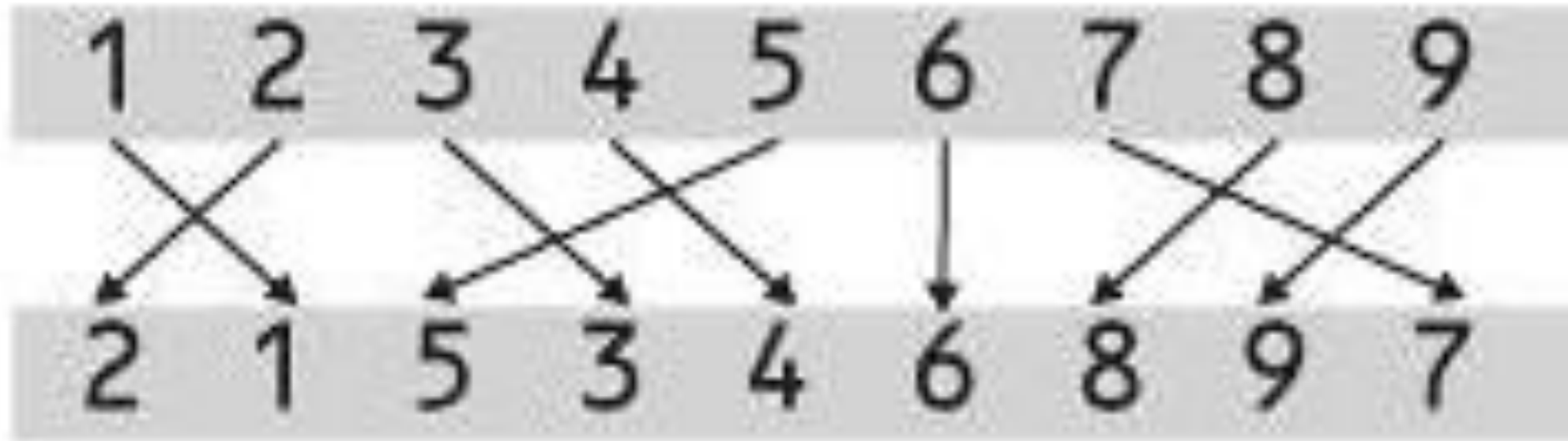


Big Idea #2: Diffusion

It's also a good idea to spread out the message. An example of this 'diffusion' is a simple column transposition:

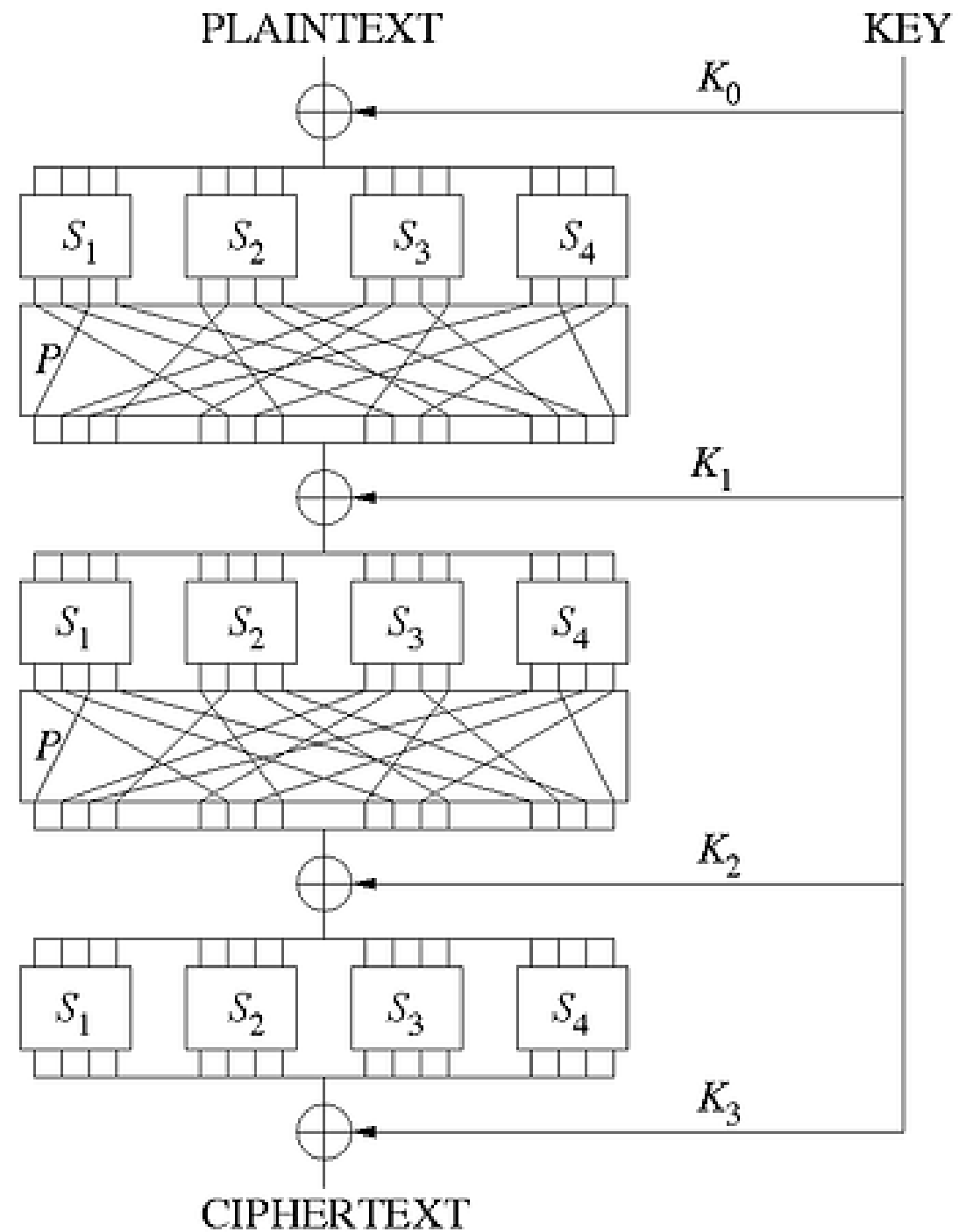


Transposition (or Permutation)



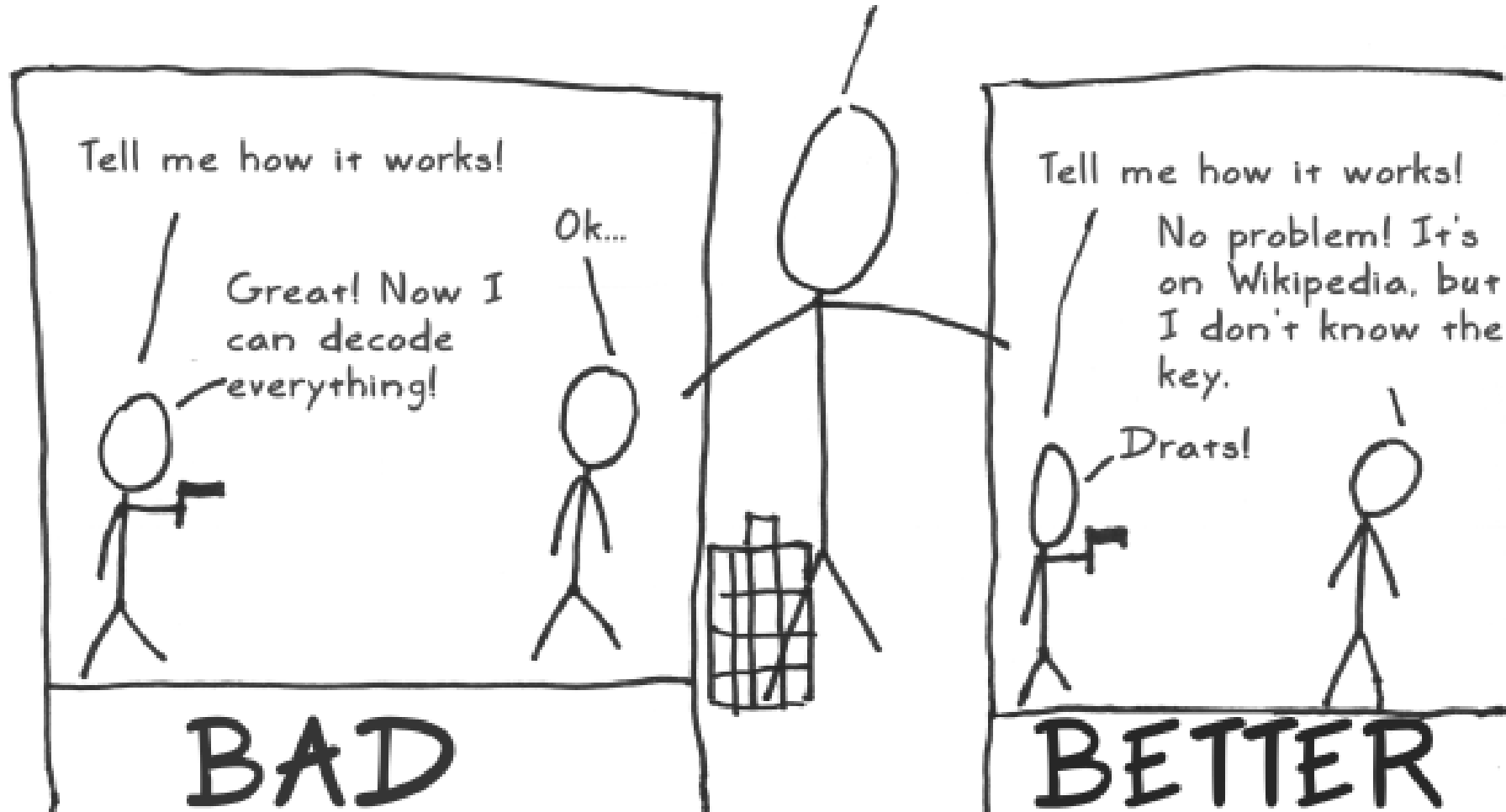
T O P S E C R E T
O T E P S C E T R

SP-Network (Substitution-Permutation)



Big Idea #3: Secrecy Only in the Key

After thousands of years, we learned that it's a bad idea to assume that no one knows how your method works. Someone will eventually find that out.





Caesar Cipher

$$E_n(x) = (x + n) \mod 26.$$

$$D_n(x) = (x - n) \mod 26.$$

Caesar Cipher

$$E_n(x) = (x + n) \mod 26.$$

$$D_n(x) = (x - n) \mod 26.$$

Ciphertext: ODVW ZHHN WKH OHFWXUHU ZDV LOO

Caesar Cipher

$$E_n(x) = (x + n) \mod 26.$$

$$D_n(x) = (x - n) \mod 26.$$

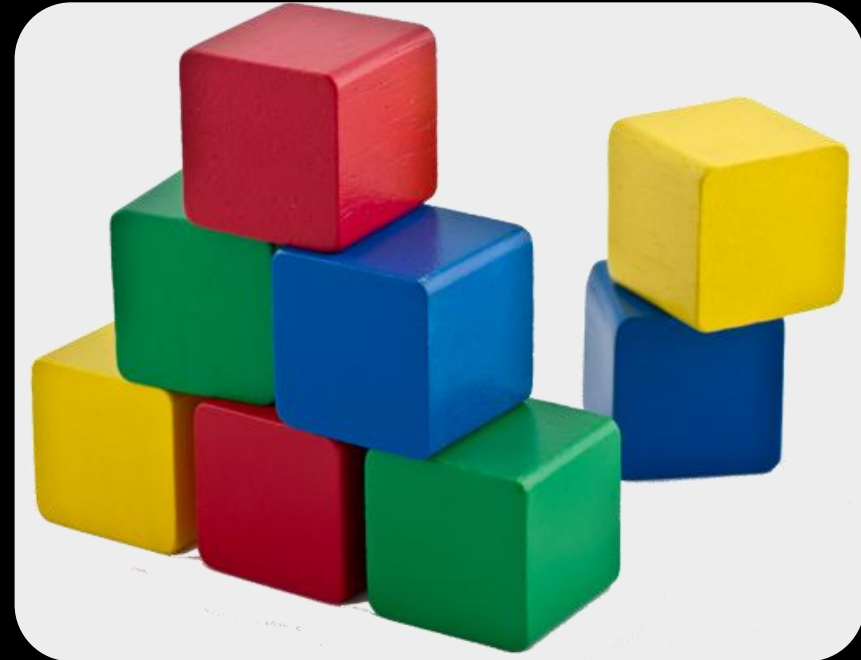
Ciphertext: ODVW ZHHN WKH OHFWXUHU ZDV LOO

Plaintext: LAST WEEK THE LECTURER WAS ILL

Stream Ciphers

and

Block Ciphers

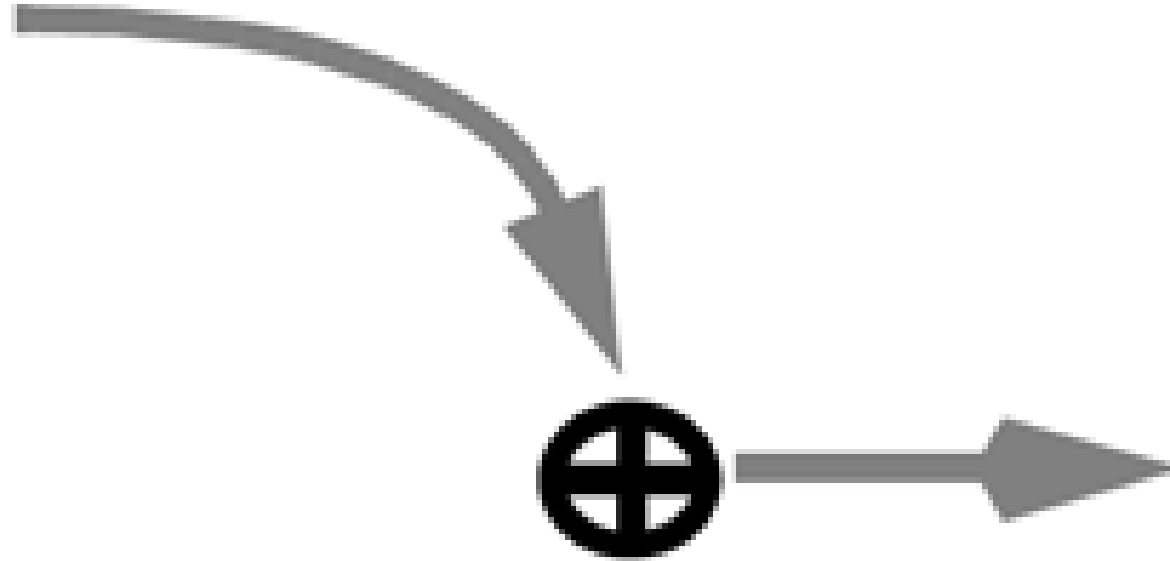


Stream Ciphers

Stream Ciphers

Key Stream
...10010101...

Plaintext
...01011001...

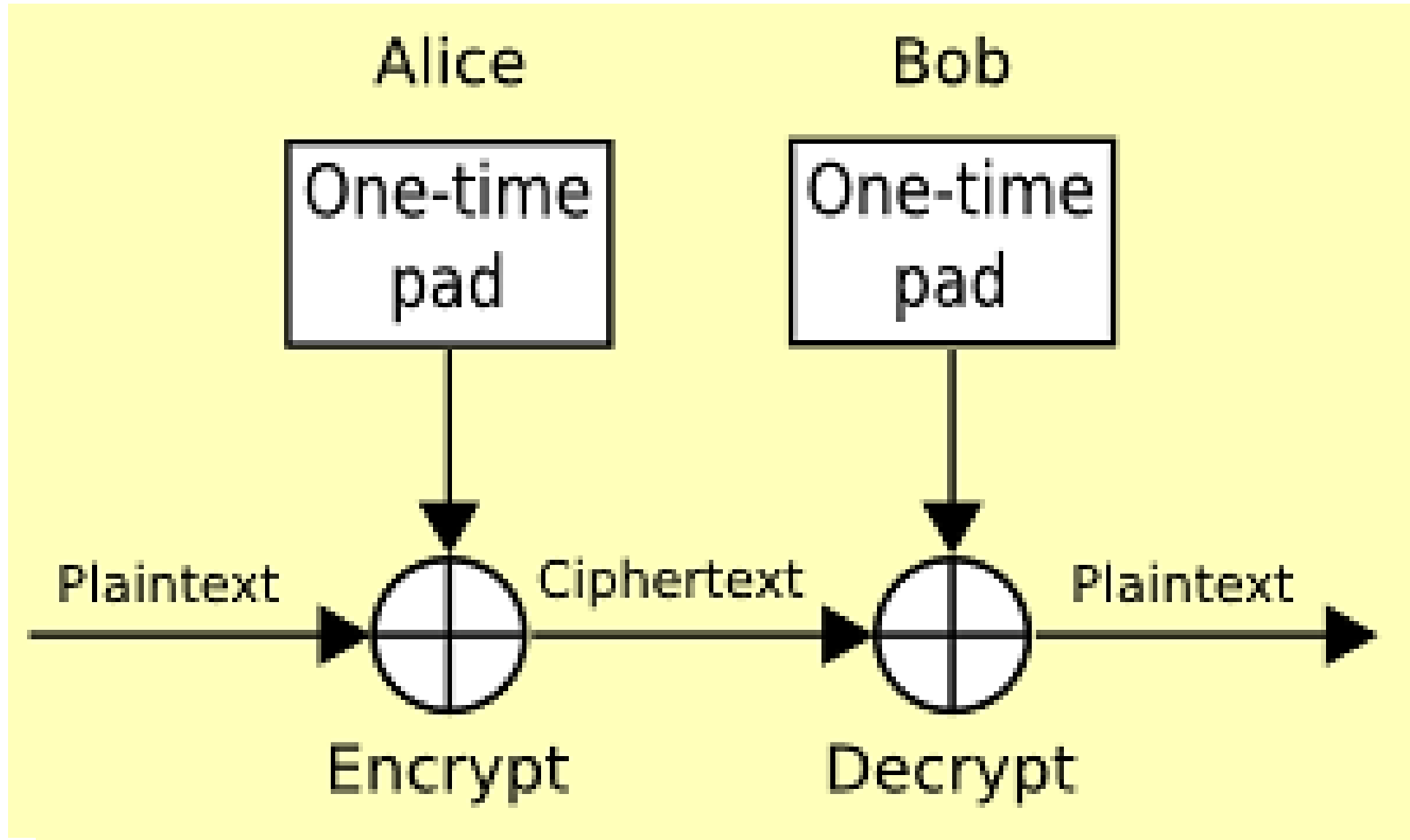


Ciphertext
...11001100...

One-time pad

- “The perfect cipher” is the only proven method for cryptography that makes an unbreakable encryption possible. Other names are the one-time pad or the Vernam scheme (Gilbert Vernam, in 1917).
- The key is **completely random** and has the **same size** as the plaintext. This makes the one-time pad impractical.
- Example:
01011 XOR 10111 = 11100 (plaintext XOR key = ciphertext)
11100 XOR 10111 = 01011 (ciphertext XOR key = plaintext)

One-time pad: Encrypt-Decrypt

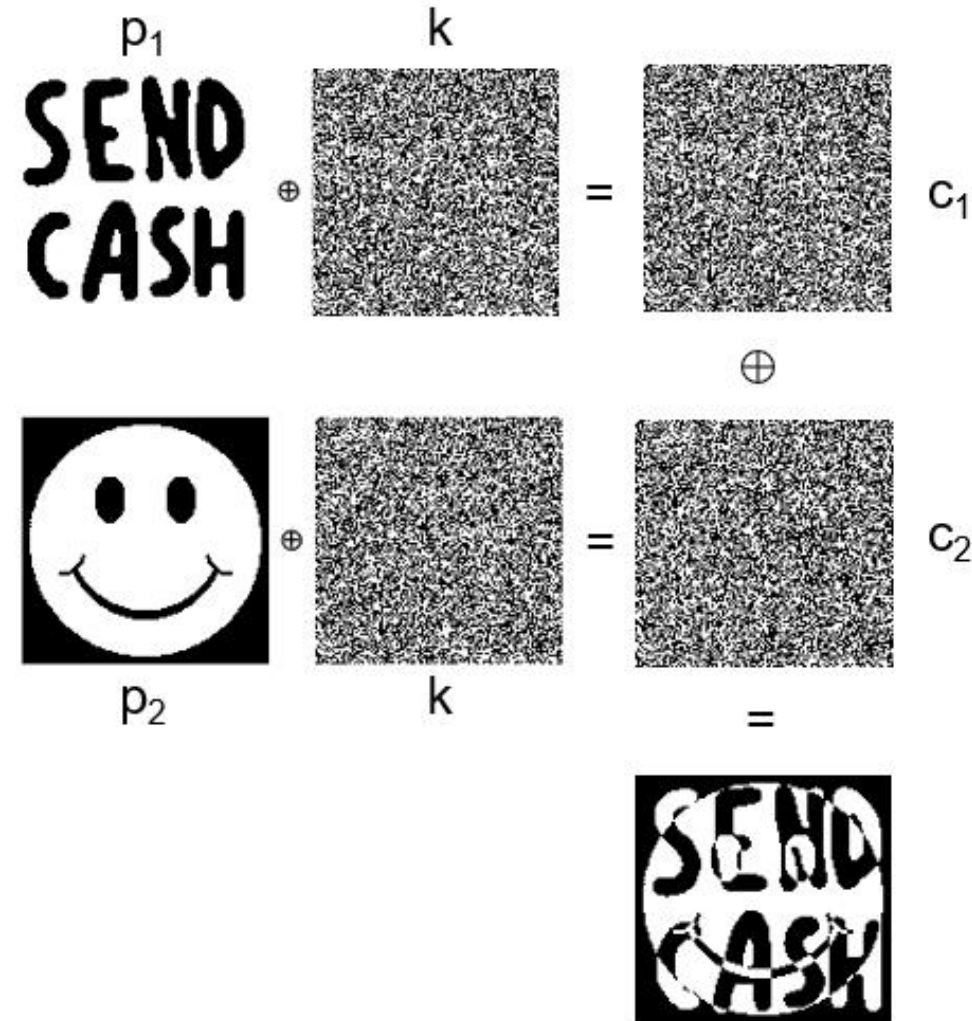


One-time pad: Re-use of the same key

- With the one-time pad, it is very important to **NEVER** use the same key more than once.

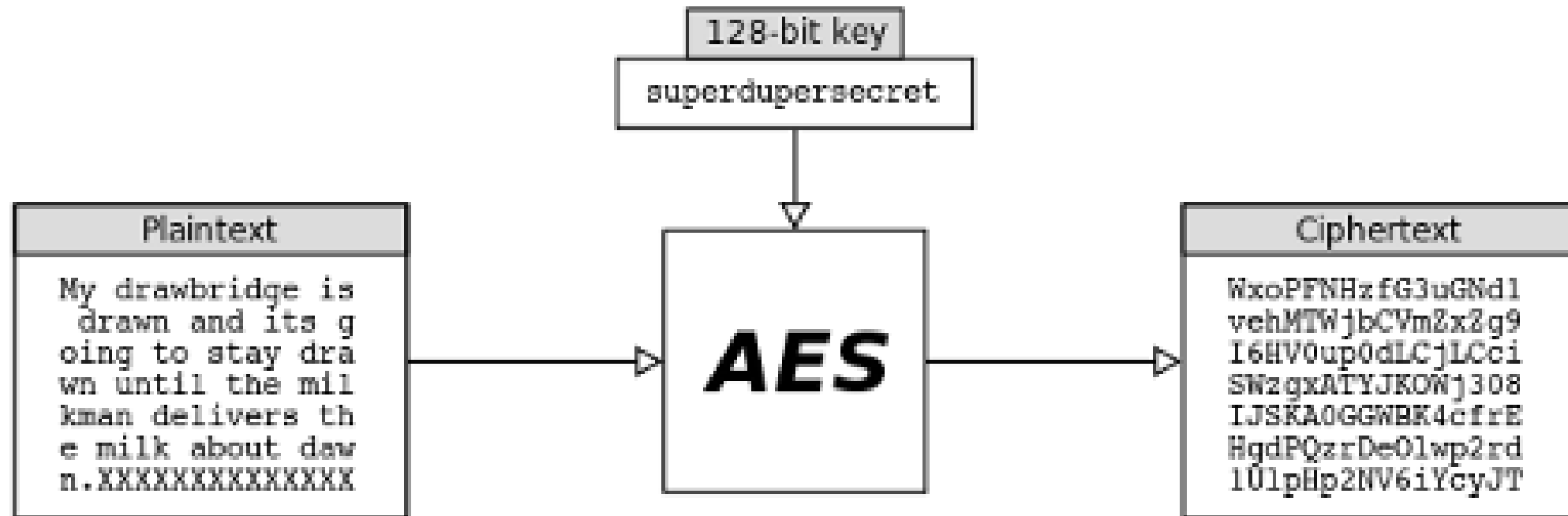
$$\begin{array}{r} c_1 = p_1 \oplus k \\ c_2 = p_2 \oplus k \\ \oplus \quad \hline c_1 \oplus c_2 = p_1 \oplus p_2 \end{array}$$

One-time pad: Re-use of the same key - visualize



Block Ciphers

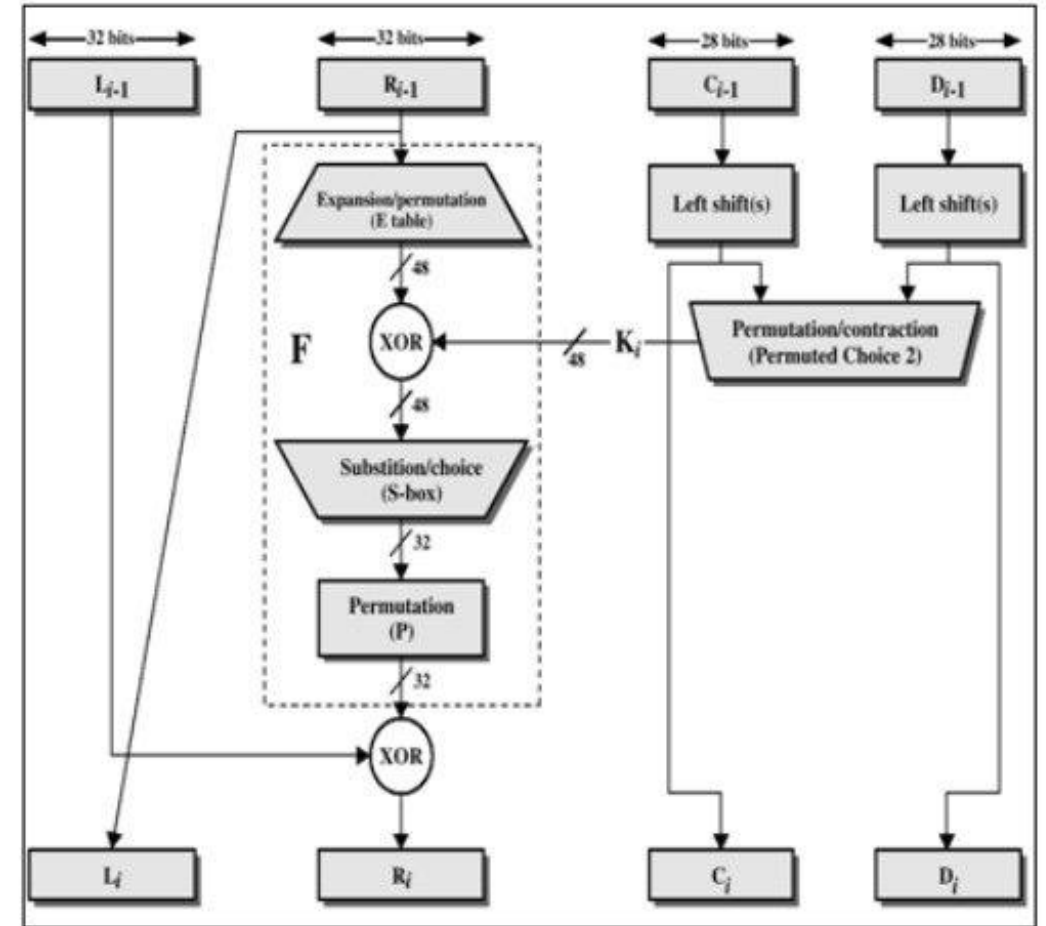
Block Ciphers



Examples of Block Ciphers (1)

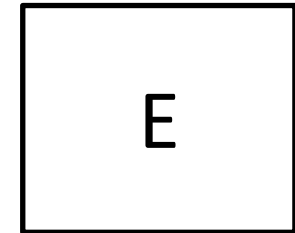
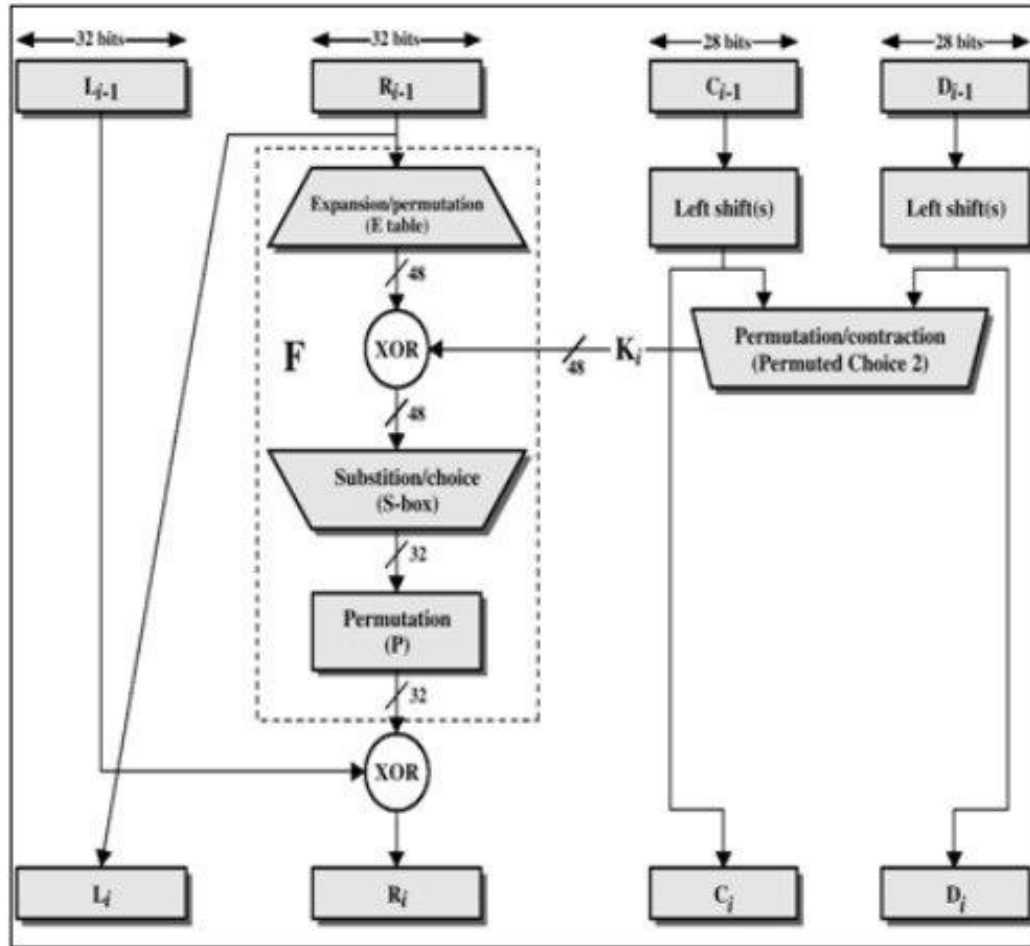
DES = Data Encryption Standard

- 1977
- Key size: 56 bits
- Block size: 64 bits
- 16-round Feistel structure
- Brute-force attack possible
- Derived version: Triple DES
 - 168-bit key (3x56 bits)
 - $2^{112} = 2^{2 \times 56}$ steps to perform a brute-force attack (meet in the middle)

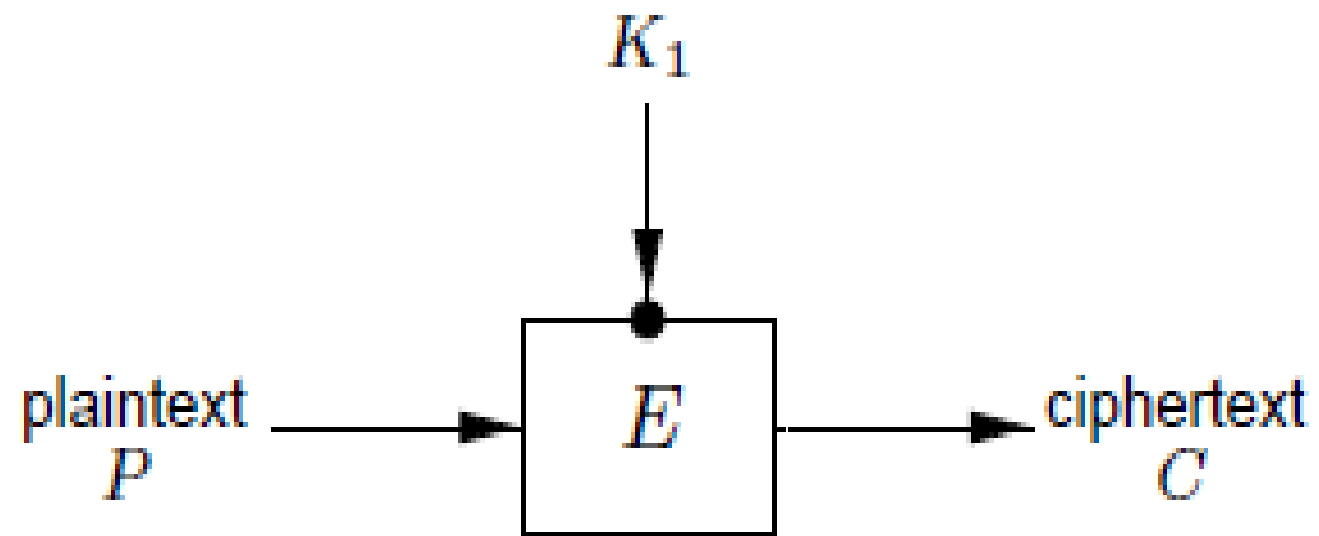


$$TripleDES_{k_1, k_2, k_3}(block) = Enc_{k_3}(Dec_{k_2}(Enc_{k_1}(block)))$$

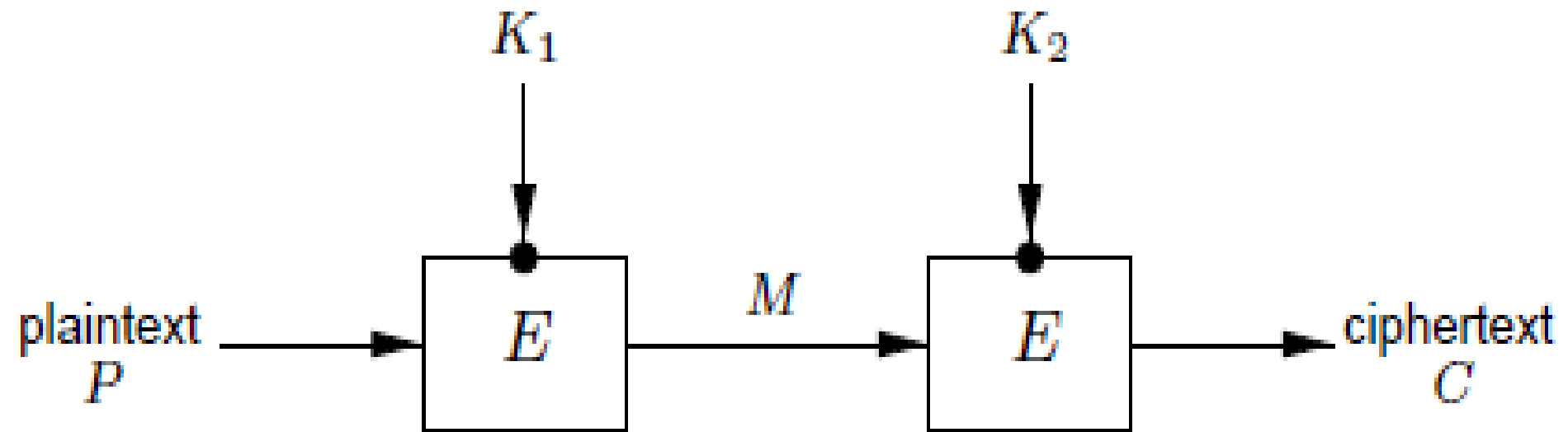
Let's simplify...



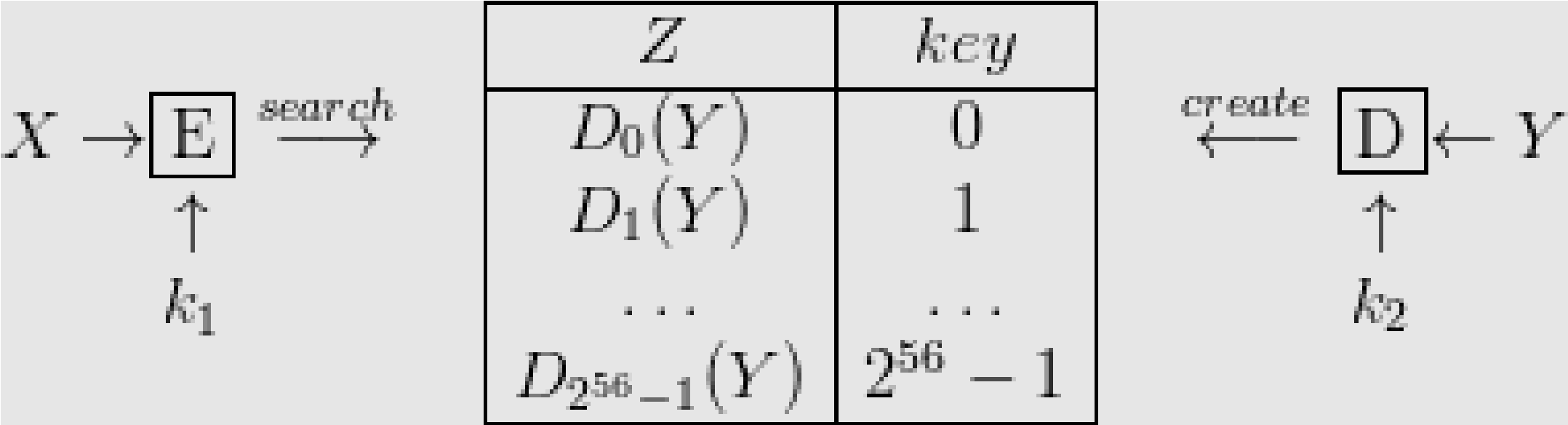
DES



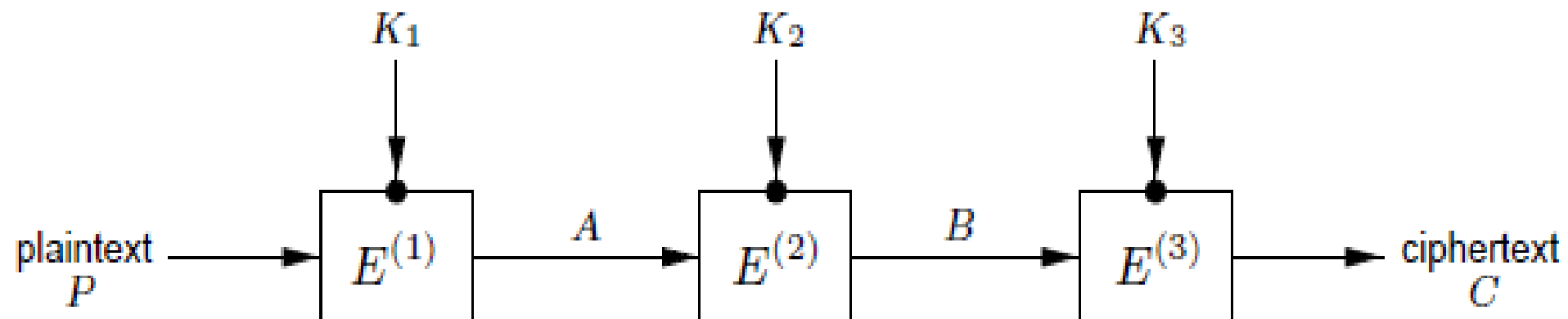
Double DES



2DES: Meet-in-the-Middle Attack



Triple DES



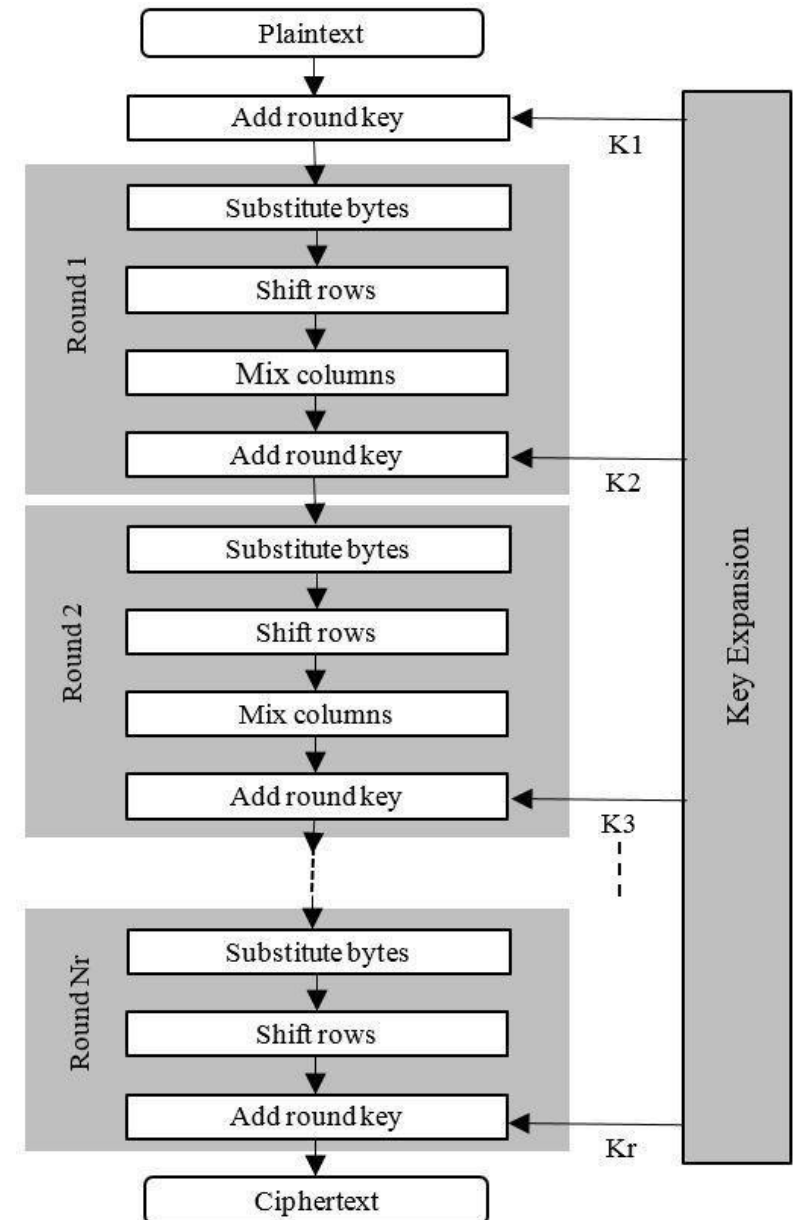
Recommended key sizes for symmetric-key crypto

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2018-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2018-2068)</i>	256	15360	512	15360	512	512

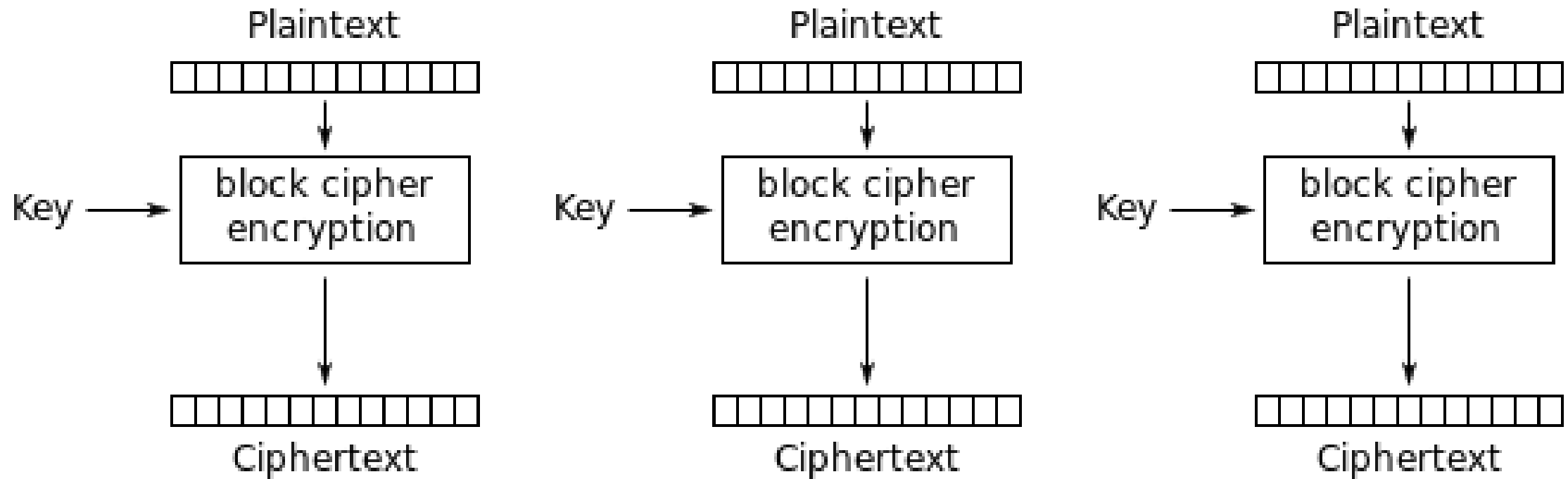
Source: keylength.com (ECRYPT-CSA recommendations, 2018)

Examples of block ciphers (2)

- AES = Advanced Encryption Standard
 - 2001
 - Key size: 128, 192 or 256 bits
 - Block size: 128 bits



Block Ciphers: Encryption Modes

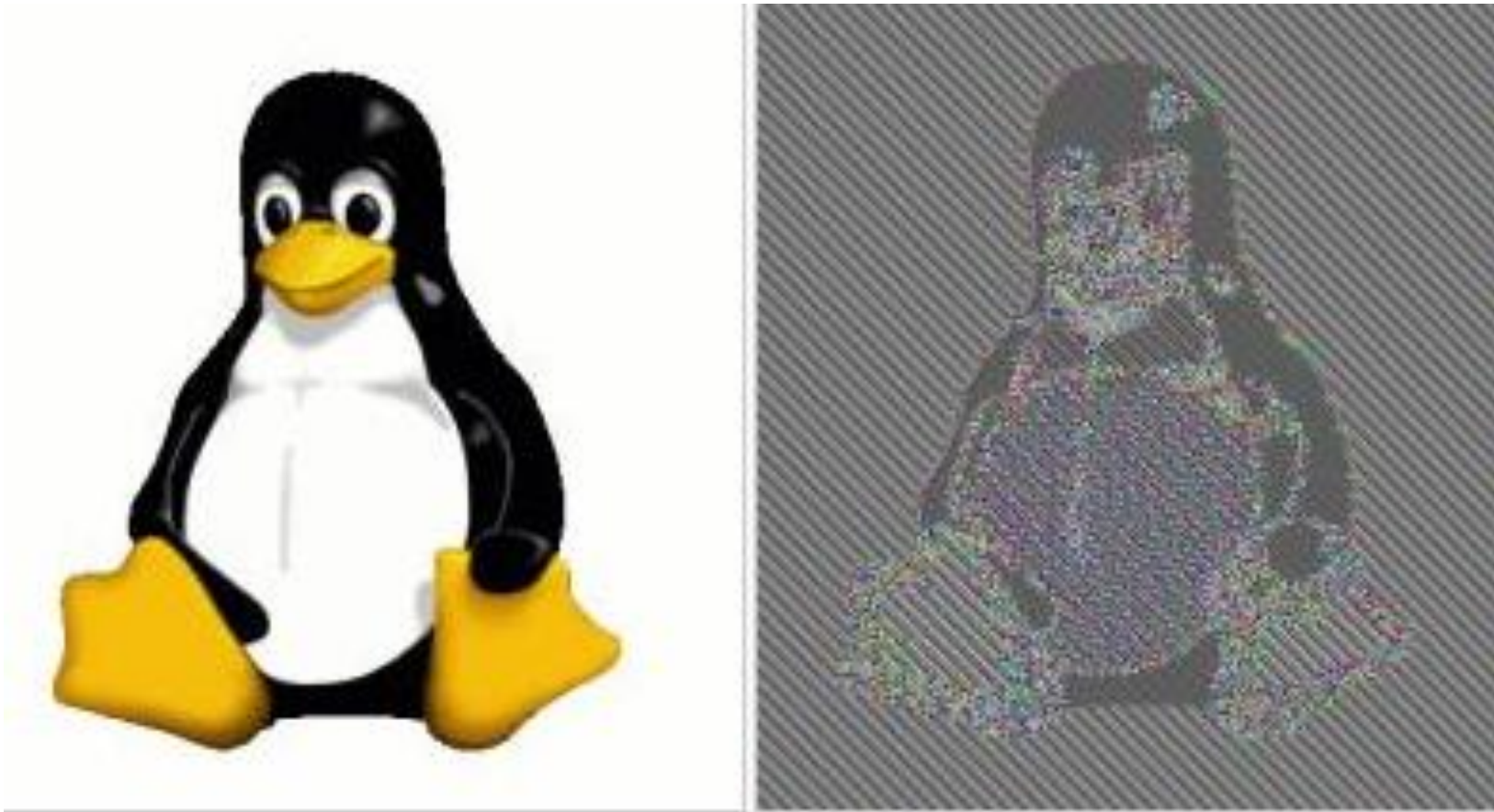


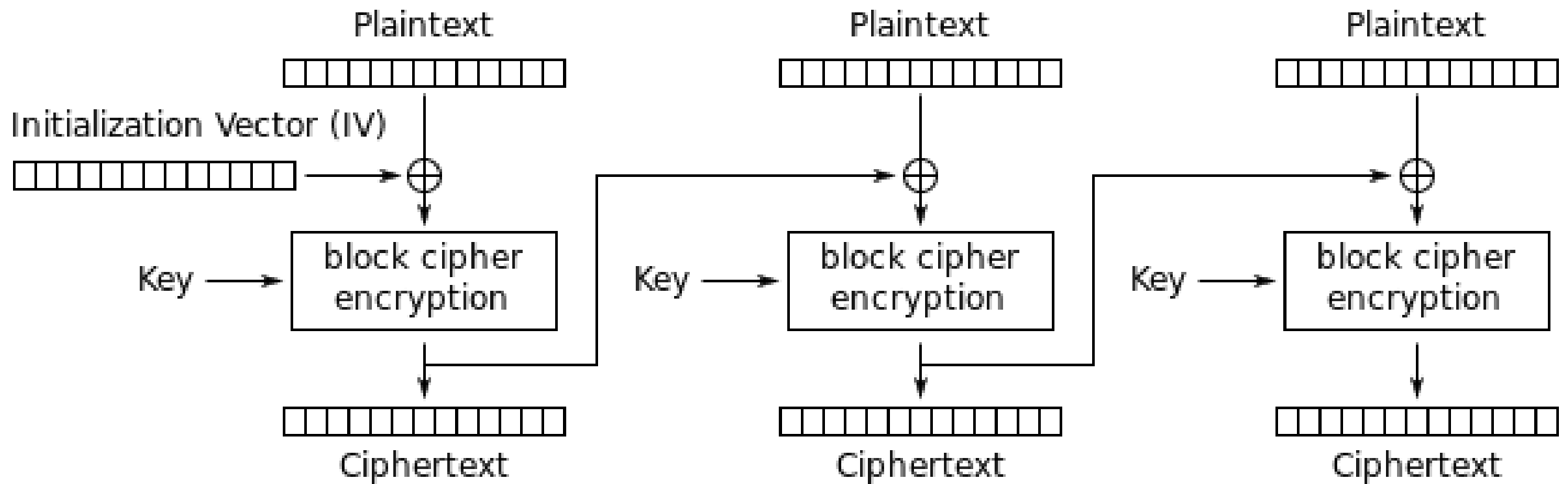
Electronic Codebook (ECB) mode encryption

Modes Effect...



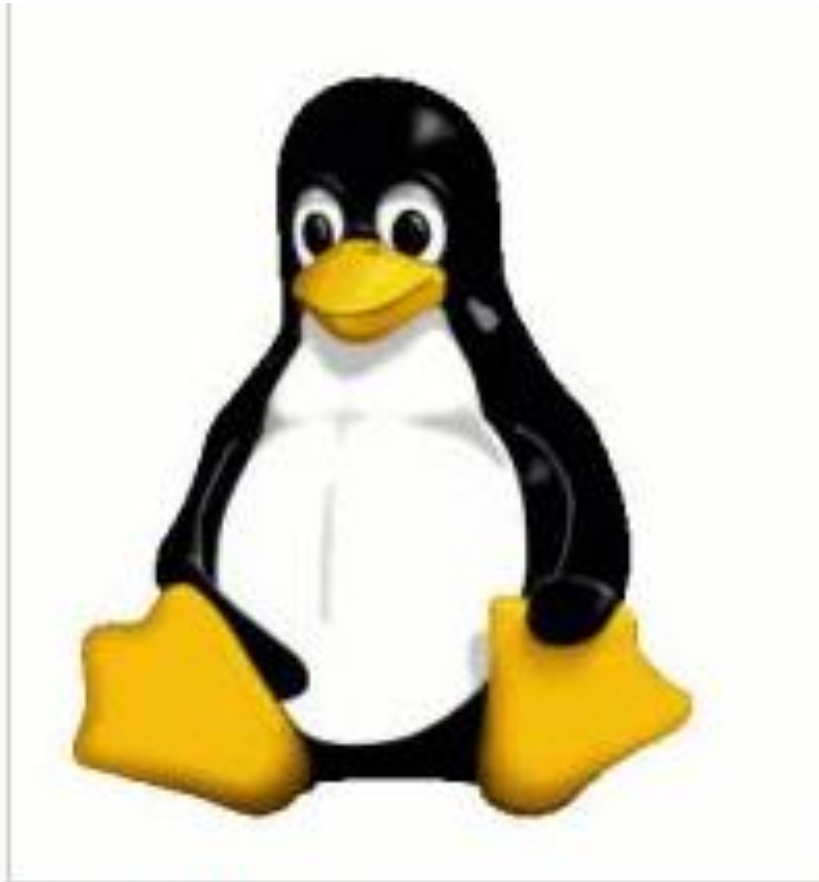
Modes Effect...





Cipher Block Chaining (CBC) mode encryption

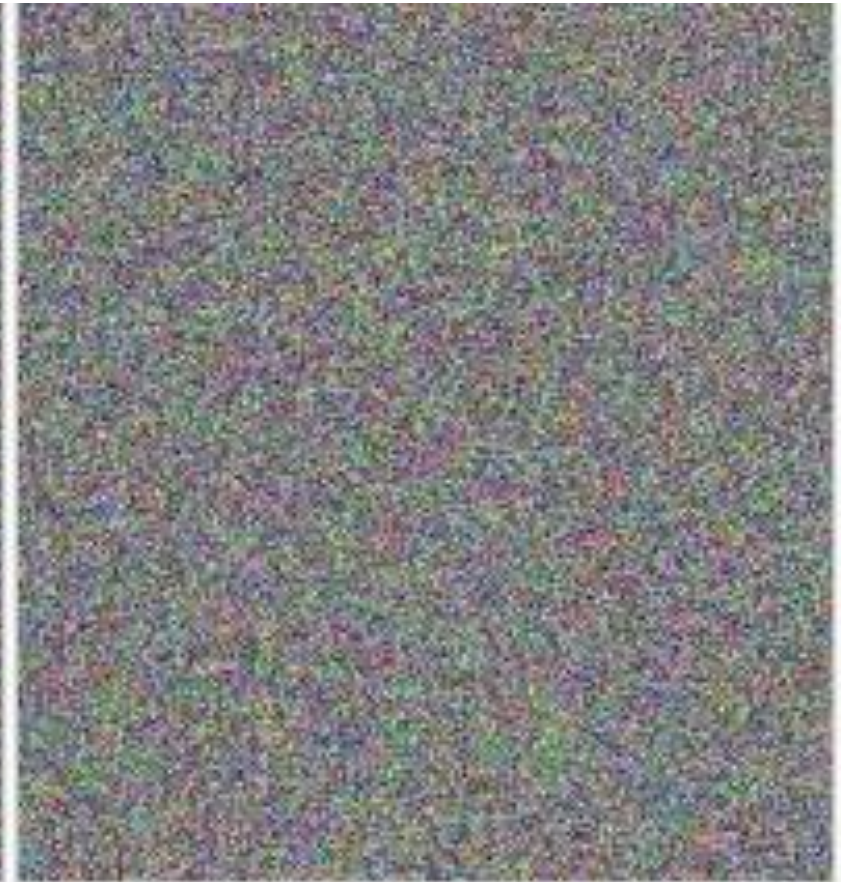
Modes Effect...



Original image

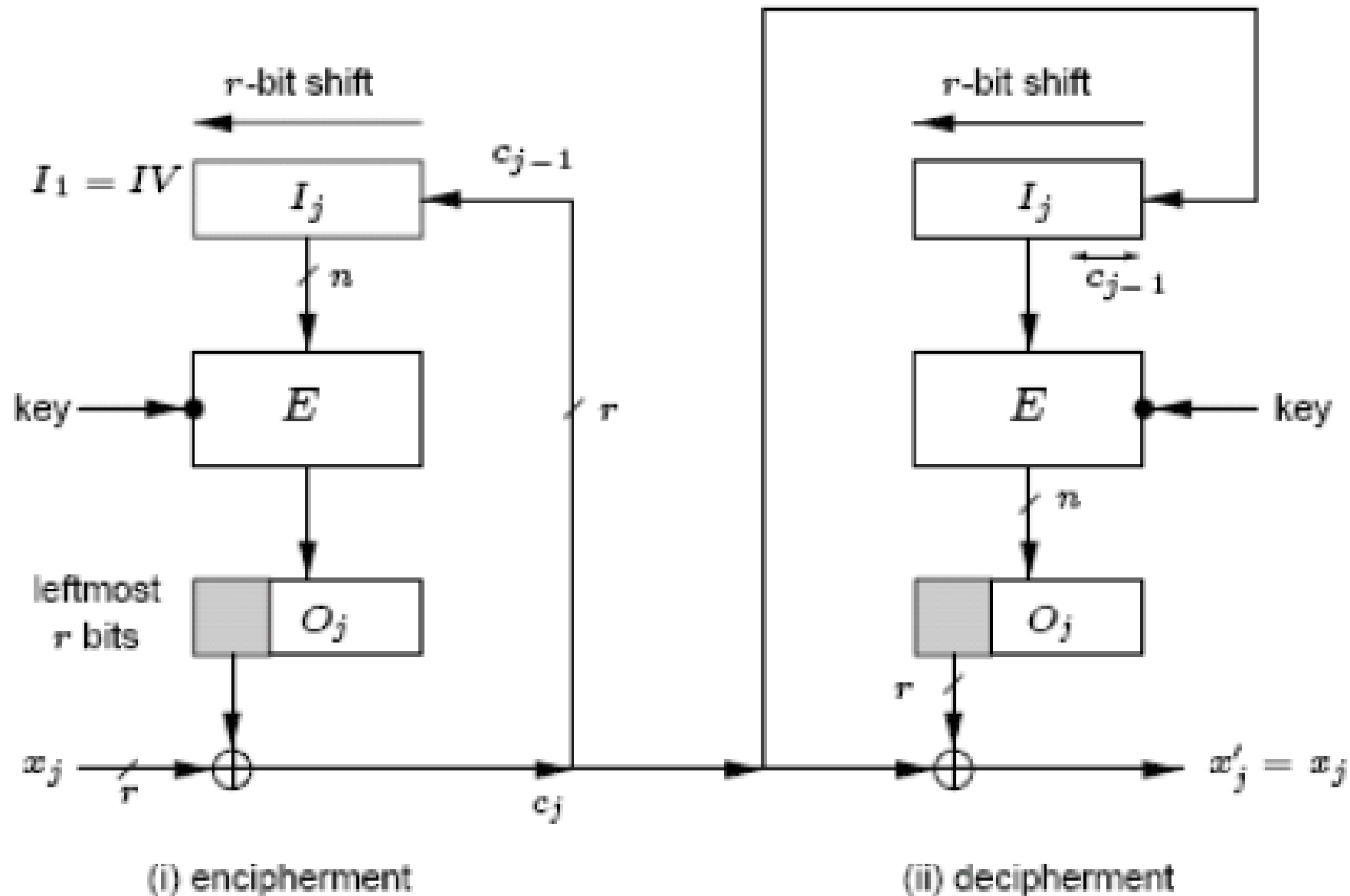


Encrypted using ECB mode

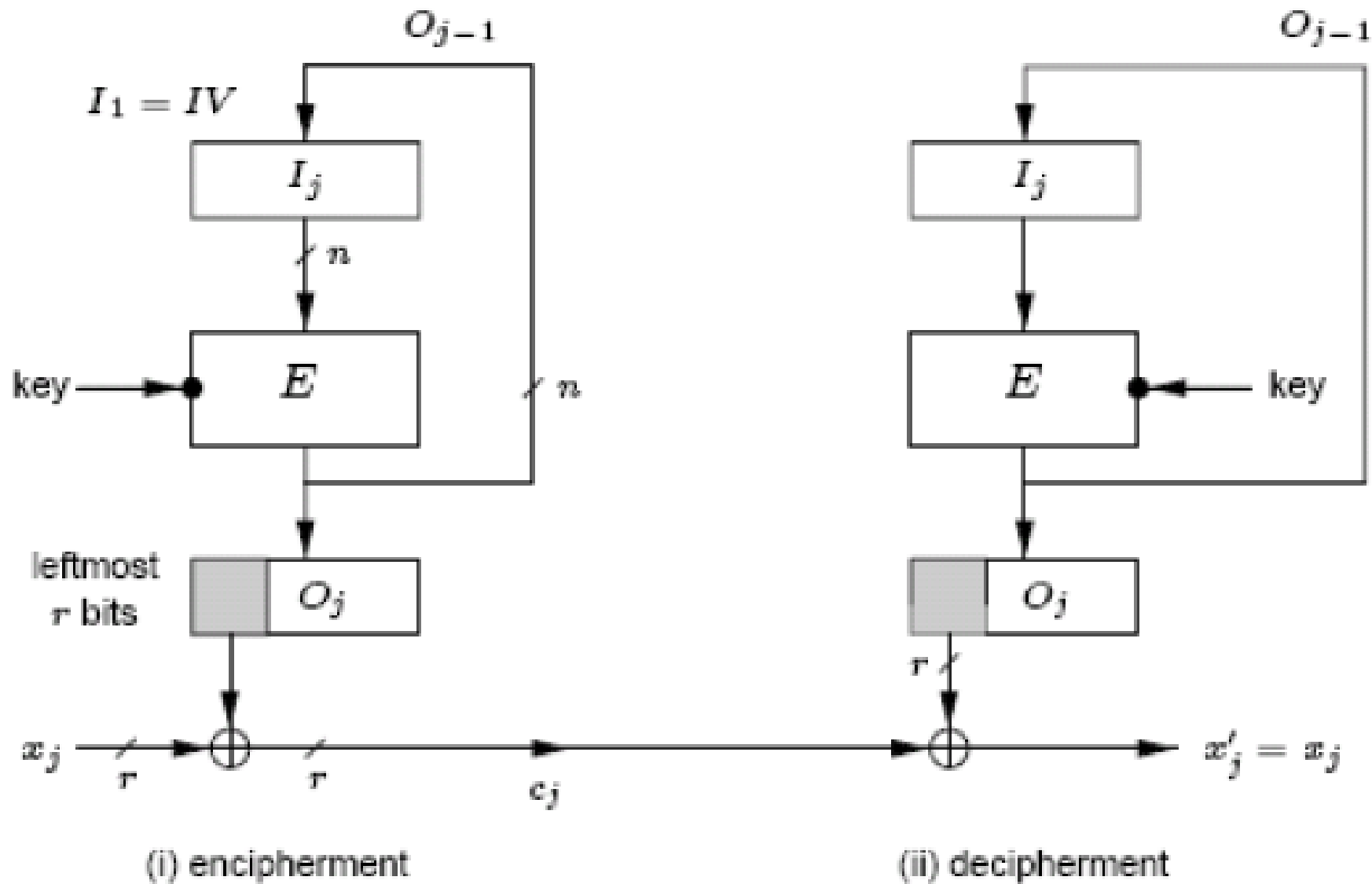


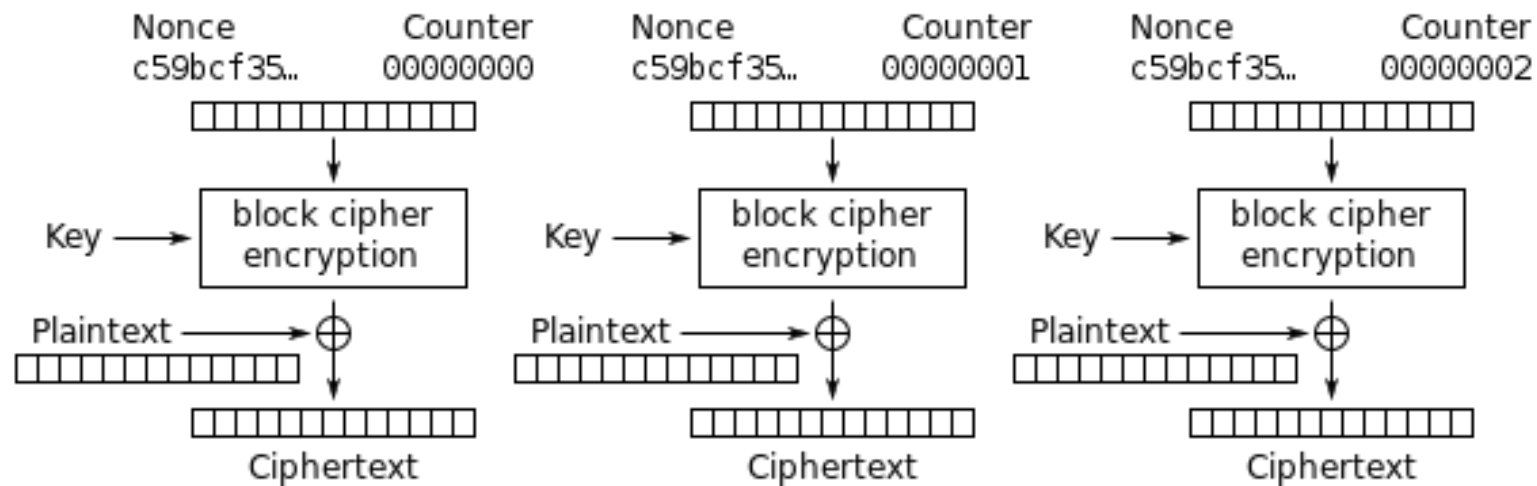
Modes other than ECB result in pseudo-randomness

Cipher feedback (CFB), r -bit characters/ r -bit feedback

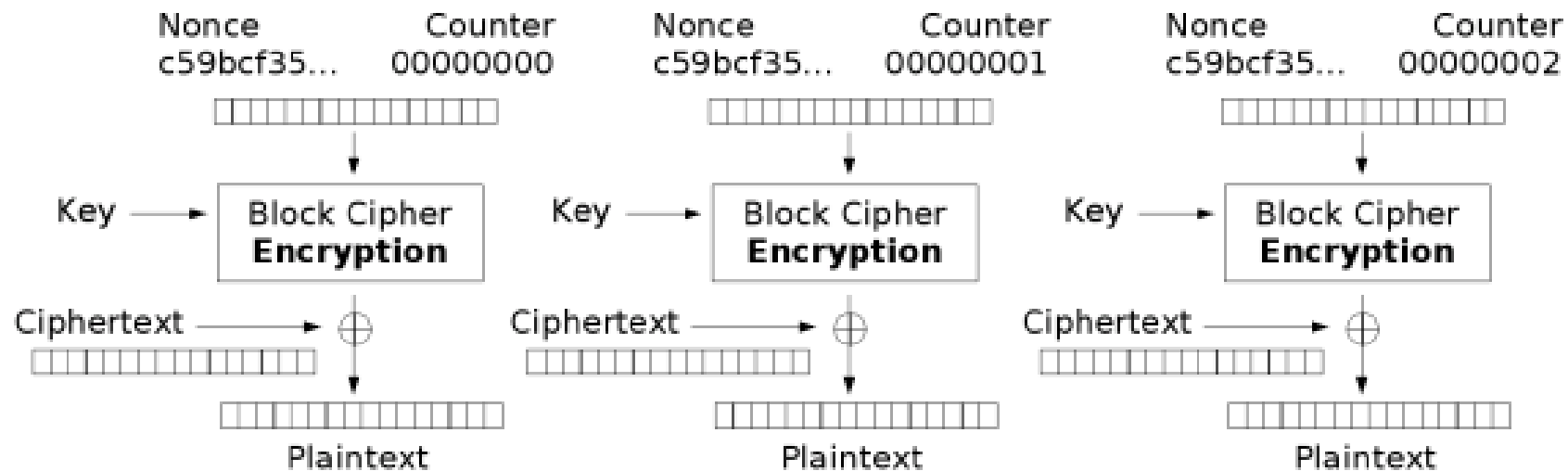


Output feedback (OFB), r -bit characters/ n -bit feedback





Counter (CTR) mode encryption



Counter (CTR) mode decryption

Properties block cipher modes of operation

- Hiding patterns in the plaintext
 - ECB mode is insecure.
 - CTR mode is insecure if the counter is reset to the same value every time.
- Error propagation
 - OFB mode and CTR mode do not propagate errors.
- Synchronisation between sender and receiver
 - CFB is self-synchronising.
- Speed
 - CFB mode is slow.
 - CTR mode is fast.

Hash Functions

Hash functions (1)

- Hash functions are functions that project a message of arbitrary length to a message of fixed length.

Original text	hello world
Original bytes	68:65:6c:6c:6f:20:77:6f:72:6c:64 (length=11)

CRC32	0d4a1185
MD5	5eb63bbbe01eeed093cb22bb8f5acdc3
SHA-1	2aae6c35c94fcb415dbe95f408b9ce91ee846ed
SHA-256	b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9
SHA-384	fdbd8e75a67f29f701a4e040385e2e23986303ea10239211af907fcbb83578b3e417cb71 ce646efd0819dd8c088de1bd
SHA-512	309ecc489c12d6eb4cc40f50c902f2b4d0ed77ee511a7c7a9bcd3ca86d4cd86f989dd35b c5ff499670da34255b45b0cfd830e81f605dcf7dc5542e93ae9cd76f
Whirlpool	6acb0f8614091b16187853927167576de1e3bae9b1302d607197f4e026ff4498d91296a3 98350faae2a612b6069d38af6cc5fcf73fd006adbeddf4cb5b84f904

Try it yourself at: <https://www.fileformat.info/tool/hash.htm>

Hash functions (2)

- Another example:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque cursus posuere neque, sit amet mollis ante. Donec pulvinar lacus vitae nulla malesuada, in aliquam dolor molestie. Duis at nulla in arcu convallis convallis. Maecenas in leo ut metus semper fermentum. Fusce orci lectus, fermentum et est at, rutrum tincidunt orci. Praesent dignissim turpis ex, eget porttitor neque porta mollis. Curabitur ac dictum turpis, eu fringilla felis. Sed semper dolor et quam facilisis mollis.

Hash functions (3)

Original text	Lorem ipsum ...
Original bytes	4c6f72656d20697073756d20646f6c6f722073697420616d65... (length=490)
CRC32	d7aca16d
MD5	baea9f76055eea8f0c162432611ead84
SHA-1	4f189835fab2bf72a4cfeaaee072201ad2bb6f489
SHA-256	40f2a4e72d43359b9567b47d9d774bf5cca1055310b32b77e1320a7c56dabffc
SHA-384	c739cf3cb6332b93b9813c83689d7b26156213bff9d2b3d7edd8d0dbad61ff7af78dec298854c19c2876bdd0133dda76
SHA-512	5cd61791060d8b26f05cab8c80556d281a98b151318295f216e622de9ad3861c6543d70733ffb56ac429269c42b44ebf40cb9b80293adc407d92ff5ac7fd4303
Whirlpool	56bac218d72c9cfc39b218c012405896bd9803dc254bfe7444c95ade63d68dbe4f8d30c1b9c73b1cdde43adacf64245e6fb426746bfc28adb359d685143a5d6a

Properties of hash functions

The ideal hash function for cryptographic applications has the following properties:

- It is **easy** to calculate the hash of a message
- Given the hash value h , it is **difficult** to find a message that has h as a hash value
= **pre-image resistance**
- Given the message m_1 , it is **difficult** to find a message m_2 that has the same hash value as the hash value of the message m_1
= **second pre-image resistance**
- It is **difficult** to find two different messages with the same hash value
= **collision resistance**

Commonly used hash functions

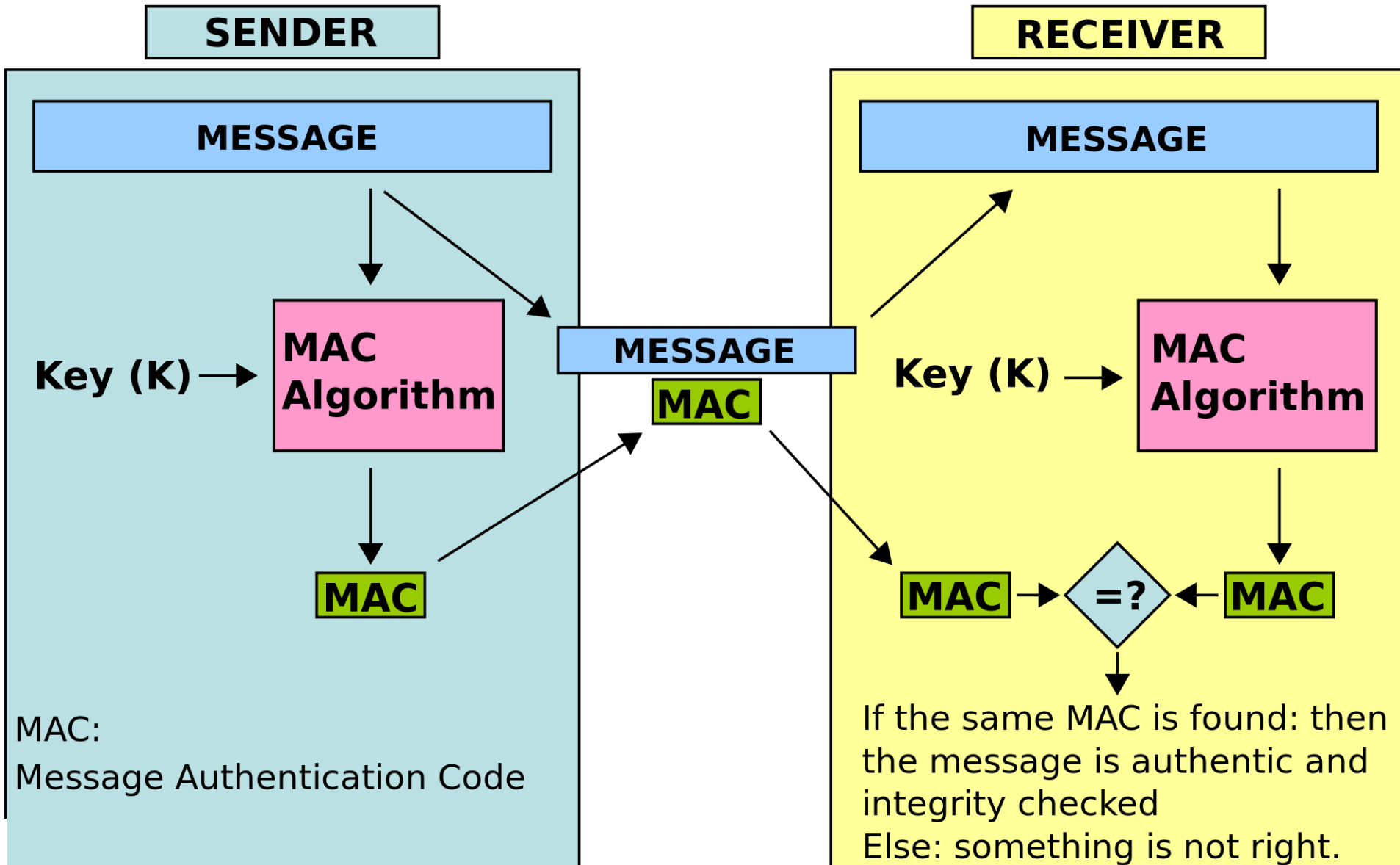
Function ↕	Year ^[gi 1] ↕	Designer ↕	Derived from ↕	Reference ↕
HAVAL	1992	Yuliang Zheng Josef Pieprzyk Jennifer Seberry		Website ↗
MD2	1989	Ronald Rivest		RFC 1319 ↗
MD4	1990			RFC 1320 ↗
MD5	1992		MD4 RFC 1321 ↗ page 1	RFC 1321 ↗
MD6	2008			md6_report.pdf 📄
RIPEMD	1990	The RIPE Consortium [1] ↗	MD4	
RIPEMD-128 RIPEMD-256 RIPEMD-160 RIPEMD-320	1996	Hans Dobbertin Antoon Bosselaers Bart Preneel	RIPEMD[2] ↗	Website ↗
SHA-0	1993	NSA		SHA-0 ↗
SHA-1	1995		SHA-0	[3] 📄
SHA-256 SHA-512 SHA-384	2002			
SHA-224	2004			
GOST R 34.11-94	1994	FAPSI and VNIIstandart	GOST 28147-89	RFC 5831 ↗ , RFC 4357 ↗
Tiger	1995	Ross Anderson Eli Biham		Website ↗
Whirlpool	2004	Vincent Rijmen Paulo Barreto		Website ↗
SHA-3 (Keccak)	2008 ^[2]	Guido Bertoni Joan Daemen Michaël Peeters Gilles Van Assche		Website ↗

Source: wikipedia

The use of a secret key

- Hash functions **without** a secret key are used to assure data integrity. They allow to detect changes in the transmitted messages.
- Hash functions can also be used for authentication (of data). In this case, a **secret key** is used, together with the message, and we call the function a Message Authentication Code (MAC).

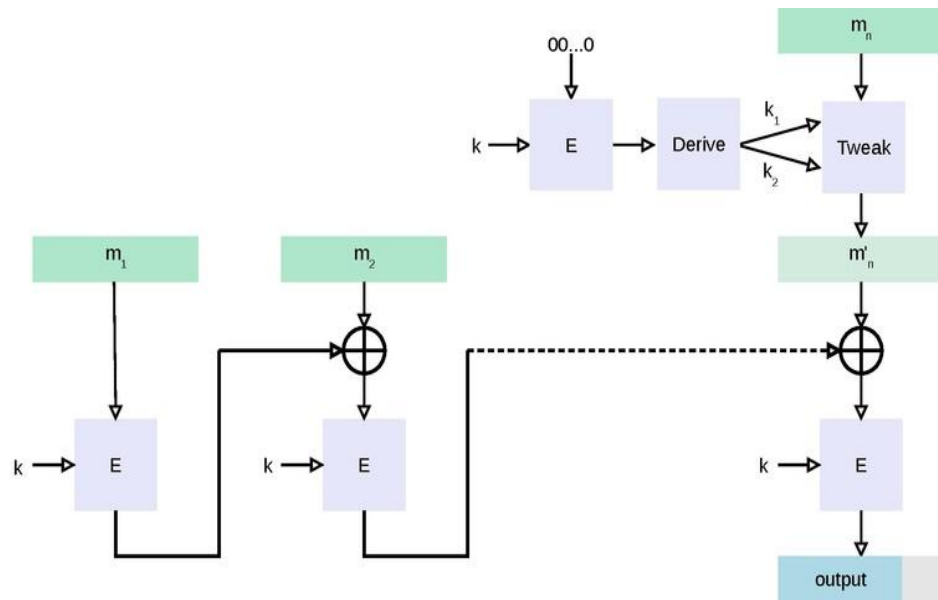
Message Authentication Code – MAC



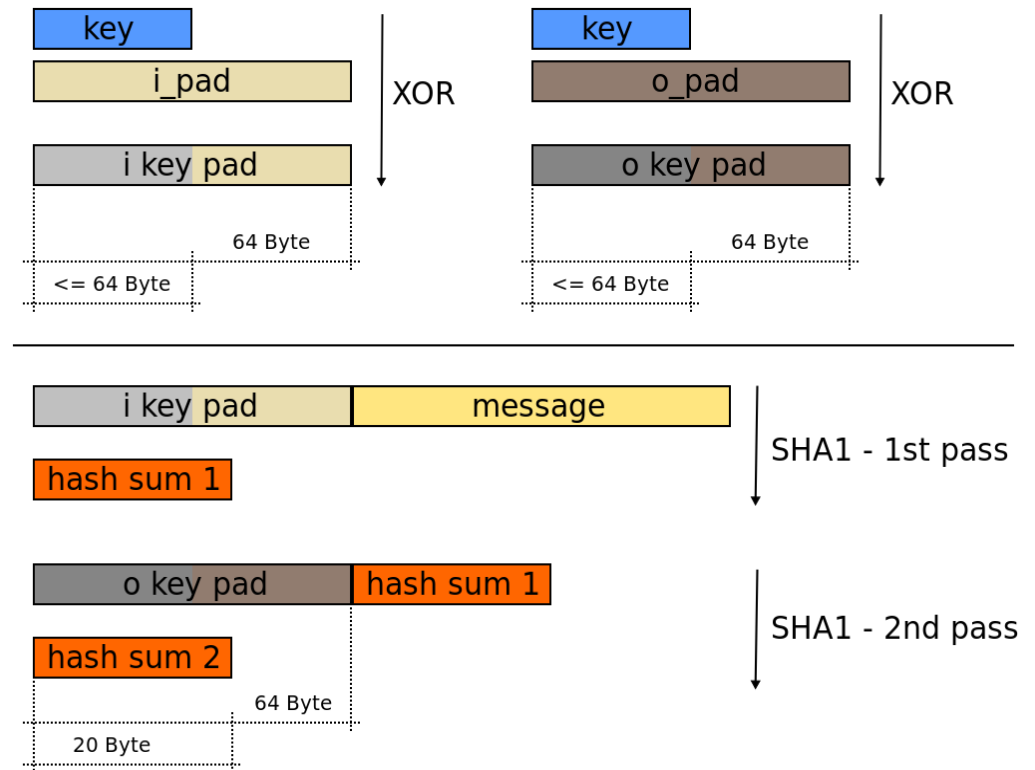
Examples of MAC algorithms

Commonly used MAC algorithms are:

- CMAC: MAC based on a symmetric-key block cipher
- HMAC: MAC based on a hash function



CMAC

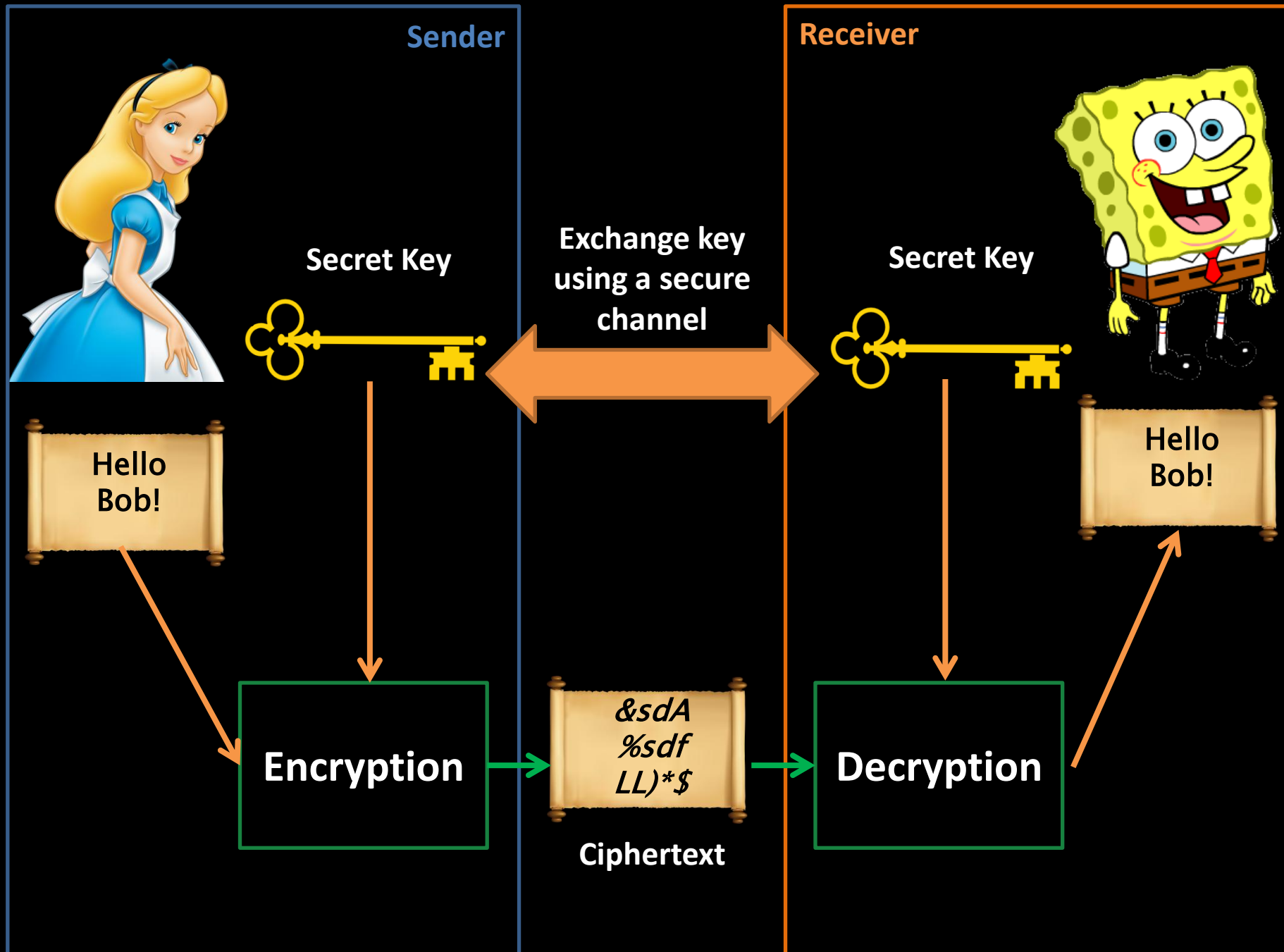


HMAC

Recommended size of the hash value

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2018-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2018-2068)</i>	256	15360	512	15360	512	512

Source: keylength.com (ECRYPT-CSA recommendations, 2018)



References

- Nigel Smart, Cryptography Knowledge Area – The Cyber Security Body of Knowledge, Available at: https://www.cybok.org/media/downloads/Cryptography_v1.0.1.pdf (Chapters 3, 4 and 5)
- Alfred J. Menezes, Paul C. van Oorschot and Scott A. Vanstone, Handbook of Applied Cryptography, Available at: <https://cacr.uwaterloo.ca/hac/> (Chapters 1, 6, 7 and 9 – the relevant parts therein)

Outlook

- We covered the basics of Symmetric Encryption
- Next week we move to **Asymmetric Encryption**
- In a few weeks we will discuss *Secure Computation Technologies*, enabled by cryptographic techniques

Stay Tuned.....

Questions?



Universiteit
Leiden

Bij ons leer je de wereld kennen